Journal of Intelligent Information Systems
https://doi.org/10.1007/s10844-025-00982-0

RESEARCH

DeepVulHunter: enhancing the code vulnerability detection capability of LLMs through multi-round analysis

Yutong Jiao[1] · **Jiaxuan Han**[1,2] · **Cheng Huang**[1]

Received: 27 May 2025 / Revised: 26 August 2025 / Accepted: 26 August 2025

Abstract

As the economic losses caused by software vulnerabilities continue to escalate, automated vulnerability detection has emerged as a crucial demand in software engineering. While current Large Language Model (LLM)-based approaches demonstrate promising capabilities for vulnerability detection, they still face significant challenges including susceptibility to non-vulnerability factors like code length, severe hallucination issues, and unsatisfactory detection accuracy and balance. To overcome these limitations, we propose DeepVulHunter, a novel multi-round detection framework that utilizes Retrieval Augmented Generation (RAG) technique to provide code snippets semantically similar to the target code and their associated vulnerability information. Extensive experiments conducted across five representative models from the Llama and Deepseek series confirm that our method effectively mitigates these challenges while enhancing both accuracy and balance in vulnerability detection tasks for general large models. The best-performing Llama-405B model achieves a detection accuracy of up to 75.3%, surpassing the current state-of-the-art approach that utilizes GPT-4 with Chain-of-Thought (CoT) prompting.

Keywords Vulnerability detection · Large language model · Retrieval augmented generation · Multi-round analysis

Yutong Jiao and Jiaxuan Han contributed equally to this work.

✉ Jiaxuan Han
zhanSxDrive30i@gmail.com

Yutong Jiao
jiaoyutong888@gmail.com

Cheng Huang
codesec@scu.edu.cn

[1] School of Cyber Science and Engineering, Sichuan University, Sichuan 610207 Chengdu, China

[2] Information Center, Guizhou Power Grid Co., Ltd., Guizhou 550003 Guiyang, China

Published online: 16 September 2025

1 Introduction

With the rapid development of computer technology, the issue of software vulnerabilities has become increasingly prominent. Research indicates that a significant number of open source code repositories contain security vulnerabilities, almost half classified as high-risk vulnerabilities (Li et al., 2021; Lipp et al., 2022). If exploited by attackers, these vulnerabilities could lead to serious security incidents and result in substantial economic losses. According to a report published by IBM in 2021 (International Business Machines, 2021), vulnerability-driven data breaches resulted in global financial losses reaching $4.35 million in 2020. This figure highlights the significant threat posed by software vulnerabilities to the global economy, and as cyberattack techniques continue to evolve, related losses are still on the rise. Therefore, there is an urgent need to develop more efficient, accurate, and intelligent code vulnerability detection methods to ensure software security and stability and to mitigate economic losses for enterprises.

Current source code vulnerability detection techniques exhibit significant limitations. Rule-based vulnerability detection methods (Gao et al., 2018; Sui and Xue, 2016) rely on predefined rules to identify vulnerabilities, their effectiveness highly dependent on the expertise and time invested by the rule developers. In addition, they demonstrate limited generalizability. Machine learning and deep learning-based approaches (Ren et al., 2025; Goud, 2025; Shang et al., 2025; Yang et al., 2024; Hou et al., 2025; Zagane et al., 2020; Al Debeyan et al., 2022; De Kraker et al., 2023; Cui et al., 2020) enhance detection capabilities through techniques such as feature extraction, graph structure learning, or natural language processing. However, the development and maintenance costs of such models are excessively high, and their poor generalization ability results in limited accuracy.

In recent years, LLMs with advanced intelligence have garnered significant attention. These models are trained on vast amounts of data with ever-increasing parameter sizes, enabling them to capture intricate patterns in natural language and demonstrate exceptional capabilities across various domains related to natural language processing: Powell and Riccardi (2025) explored using LLMs to generate human-understandable explanations for decisions in automatic scheduling systems, verifying the accuracy and scalability of the method; Raihan et al. (2025) evaluated the ability of LLMs to generate code and answer questions in computer science education, while also discussing potential impacts and misuse risks. Source code, as a form of language, can be comprehended and processed by LLMs, which has led to the emergence of LLM-based techniques for source code vulnerability detection. Cao and Jun (2024) developed a vulnerability detection method for source code that uses Common Weakness Enumeration (CWE) names and descriptions, thus improving the precision of vulnerability detection and classification. Zhang et al. (2024) enhanced the performance of LLMs in vulnerability detection tasks by incorporating source code data flow and Application Programming Interface (API) call sequences into input prompts, while also exploring the impact of CoT prompting on LLMs' vulnerability detection capabilities. Furthermore, Wen (2024) proposed a multi-agent framework, where agents are assigned distinct roles, which significantly improved the performance of LLM-based vulnerability detection systems. Liu et al. (2025) extracted feature sets from Abstract Syntax Trees (AST) and Control Flow Graphs (CFG), subsequently fine-tuning LLM to significantly enhance their vulnerability detection capabilities. Lu et al. (2024) integrated graph-structured infor-

mation extracted from code with similar code snippets into the prompt, thereby enhancing the accuracy of LLMs in vulnerability detection.

These studies collectively demonstrate the potential of large language models in enhancing the precision and efficiency of vulnerability detection, providing new perspectives and methodologies for research and applications in software security. However, Guo et al. (2024) pointed out that the ability of LLMs to detect vulnerabilities is highly dependent on their training data, with larger models showing more stable performance in terms of accuracy. Our research has found that the results of vulnerability detection using general-purpose large models (such as the Llama and DeepSeek series) are often influenced by non-vulnerability-related factors like code length (as shown in Fig. 1). Taking 100 lines as the boundary, we calculated the relative proportions of false positives (FP) and false negatives (FN) in short and long functions misclassified by different models, as shown in Fig. 2. The experimental results demonstrate that the Llama series displays a significantly higher misclassification rate for long functions as "Vulnerable" compared to longer functions, whereas the Deepseek series exhibits the opposite tendency in its classification behavior. Notably, among all models except Deepseek-R1, the relative variance attributable to code length (as evidenced by the spacing between blue-green dividing lines in each group) ranges from 5.6% to 15.5%. This empirically demonstrates that vulnerability detection outcomes are susceptible to code length when CoT prompting is not employed, thereby compromising the model's output accuracy. In addition, the training data may lead to the hallucination problem of the model itself (Ye et al., 2023), which means that the model generates incorrect or nonsensical outputs. In addition, several existing techniques for improving the performance of LLM-based vulnerability detection have several limitations, including imbalanced detection results, and low detection accuracy.

This study addresses these challenges by proposing DeepVulHunter, a novel multi-round analysis framework that utilizes RAG to provide code snippets semantically similar to the target code and their associated vulnerability information. Our method is divided into five modules: (1) Vulnerability Database Construction: build a database that contains source code, vulnerability details, and source code vectors. (2) Similar Code Retrieval: extract similar code and its associated vulnerability information from the database. (3) Similar Code

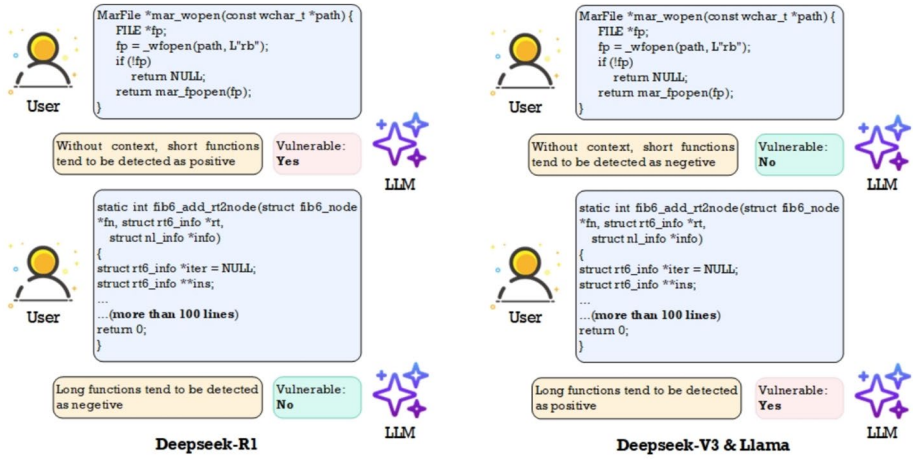

Fig. 1 An example of bias in LLM vulnerability detection caused by code length

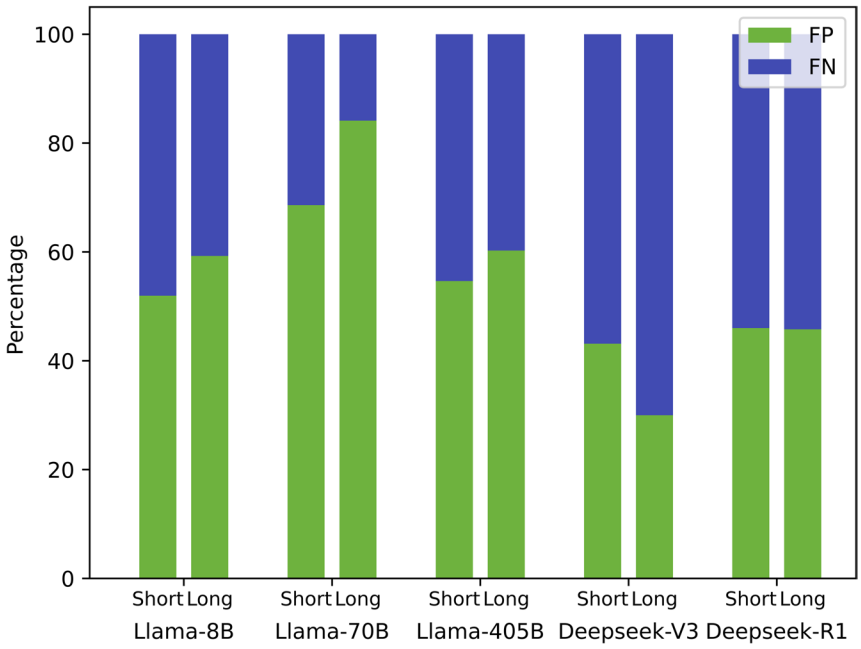

Fig. 2 Relative proportions of FP and FN in short and long functions misclassified by different models in base test. A greater distance between the blue-green boundaries indicates more severe interference from code length

Analysis: generate an analysis report on the robustness or vulnerability points of the similar code. (4) Target Code Analysis: generate a vulnerability analysis report for the target code based on the vulnerability analysis report of the similar code. (5) Analysis Validation: use the inherent vulnerability detection capabilities of LLMs for further validation when the conclusion of the target code is “no vulnerability".

In summary, the main contributions of this study are as follows:

- We propose a vulnerability detection method named DeepVulHunter based on multi-round analysis. It addresses the significant challenges faced by current LLM-based approaches for vulnerability detection, such as severe hallucination issues, and unsatisfactory detection accuracy and balance, achieving the effect of improving the accuracy and balance of LLM-based vulnerability detection.
- During the multi-round analysis process of this framework, RAG technique is introduced to provide code snippets semantically similar to the target code and their associated vulnerability information. By leveraging this vulnerability information of code semantically similar to the code to be detected, it realizes vulnerability analysis that focuses more on the logic of the target code itself, thus solving the problem of LLMs overly focusing on non-vulnerability-related factors like code length.
- A comprehensive evaluation of DeepVulHunter is conducted, demonstrating its practical value from dimensions such as accuracy and balance. The accuracy reaches as high as 75.3%, surpassing the current state-of-the-art approach that utilizes GPT-4 with CoT prompting.

2 Related work

2.1 Large language model

LLMs, which are rooted in the core concept of language modeling (LM), have recently triggered fundamental transformations in the field of artificial intelligence, demonstrating exceptional cross-domain application capabilities (Bommasani et al., 2021; Brown et al., 2020; Srivastava et al., 2022). LLMs leverage LM to model the generation probability of word sequences, thereby enabling predictions of future tokens. Unlike traditional language modeling approaches, LLMs are distinguished by their massive parameter scales and extensive pre-training on large datasets, significantly enhancing their expressive power and generalization performance. The versatility and adaptability of LLMs are widely attributed to their unprecedented billion-scale parameter sizes (Zhao et al., 2023). Currently, a variety of advanced LLMs have emerged, including but not limited to OpenAI's GPT (Achiam et al., 2023), Google's PaLM and Bard (Anil et al., 2023; Chowdhery et al., 2023), and DeepMind's Chinchilla (Hoffmann et al., 2022).

Notably, while some LLMs are exclusively accessible via APIs, others are made available to the public in open-source form (Luo et al., 2023b). In the open-source domain, Tsinghua University developed the GLM-130B model (Zeng et al., 2022), while EleutherAI contributed the GPT-NeoX-20B and GPT-J-6B models (Chen et al., 2023). Meta developed the OPT (Zhang et al., 2022) and Llama (Touvron et al., 2023) series, and Google released the UL2-20B model (Tay et al., 2022). Additionally, the recently introduced Deepseek series (Guo et al., 2025; Liu et al., 2024) has also garnered significant attention. These open-source models provide abundant research and application resources for both academia and industry.

2.2 Prompt engineering and RAG

Prompt Engineering is a technique that optimizes the performance of parameterized memory by carefully designing input prompts. It guides the model to generate high-quality, highly accurate output results that meet the task requirements by carefully formulating input prompts. Prompt engineering comprehensively utilizes strategies such as explicit instructions, context information, multi-step guidance, formatted output, and example-driven approaches to optimize the model's response. With prompt engineering, users can more efficiently tap into the potential of large language models and generate high-quality content that meets specific requirements, thus improving the accuracy and execution efficiency of tasks. Research shows that well-structured prompts can significantly improve the performance of large language models in downstream tasks (Pitis et al., 2023), which has also given rise to a variety of prompt design paradigms (Liu et al., 2023). In recent years, scholars have conducted in-depth research on the mechanism of prompts in generative models: Liu and Chilton (2022) analyzed the significant impact of different prompt keywords on the quality of image generation through experiments; Maddigan and Susnjak (2023) further explored the application of prompts in assisting large language models to generate visual results; White et al. (2024) systematically designed a variety of prompt strategies for code generation tasks, significantly improving the model's performance. The latest research has begun to focus on the application potential of prompt engineering in the field of software vulnerability detection by LLMs. Cao et al. (2025) innovatively designed an enhanced prompt

template and successfully applied ChatGPT to the task of deep-learning program repair; White et al. (2024) systematically explored a variety of prompt modes and provided optimization methods for processes such as requirement acquisition, rapid prototype development, code quality assessment, and deployment testing.

RAG (Lewis et al., 2020) is a mainstream paradigm of prompt engineering. When performing a specific task, the RAG system first extracts task-related information from the prebuilt non-parameterized memory, and then uses this information together with the task itself as a prompt. This enables the large language model to make analysis and decisions based on both its parameterized memory and non-parameterized memory. Applying RAG technology can effectively alleviate the common "hallucination" phenomenon of large language models (i.e., the information generated by the model is not based on accurate and real data but from its own "imagination" (Zhang et al., 2023b)). Large language models equipped with RAG technology perform excellently in knowledge-intensive natural language generation tasks. This technology can significantly reduce the possibility of the model generating incorrect or misleading content by accessing and integrating verified external information sources.

2.3 Vulnerability detection

2.3.1 Traditional vulnerability detection

Traditional vulnerability detection typically relies on feature engineering and traditional machine learning techniques, involving the processes and results of static code analysis, dynamic code analysis, and hybrid static-dynamic code analysis. During the early stages of traditional vulnerability detection, some studies (Gao et al., 2018; Sui and Xue, 2016; Kaur and Nayyar, 2020) utilized rules based on expert experience as templates for vulnerability detection. Grieco et al. (2016) proposed VDiscover, integrating static and dynamic features using logistic regression, random forests, and multilayer perceptrons. Shar et al. (2014) extracted results from hybrid code analysis and trained them in supervised and semi-supervised models to detect common web vulnerabilities such as SQL injection. Scandariato et al. (2014) treated software functions as a series of terms with related frequencies and extracted them as features for vulnerability detection using a bag-of-words representation. Harer et al. (2018) proposed methods for vulnerability detection using traditional models such as deep neural networks (DNN) and random forests. Pang et al. (2015) combined N-gram analysis with feature selection algorithms, defining features as sequences of consecutive tokens in source code files for vulnerability detection.

2.3.2 Deep learning based vulnerability detection

Compared with traditional methods, deep learning (DL) methods can automatically capture features and simplify the cumbersome feature extraction process. In recent years, DL has been widely applied across various domains. Existing DL-based methods can be categorized into two main types: sequence-based methods and graph-based methods.

Sequence-based methods employ DNNs to model code entities and represent code in various forms. Cheng et al. (2022) proposed the ContraFlow method, which employs a pretrained value-flow path encoder for path-sensitive code embedding. They designed a specialized value-flow path selection algorithm and conducted feasibility analysis to filter value

sequence paths, using them as input for vulnerability detection. Additionally, some studies (Qin and Eisner, 2021) have utilized convolutional neural networks (CNNs) to encode sequential code data for function-level vulnerability detection.

Graph-based methods focus on utilizing DL techniques to model program graph data, encompassing various graph structures such as CFG, data flow graphs (DFG), program dependency graphs (PDG), and AST. Li et al. (2021) employed multiple types of recurrent neural networks (RNNs), including Bi-LSTM and Bi-GRU (Bidirectional Gated Recurrent Unit), to model CFG, DFG, and PDG for software vulnerability detection. Hin et al. (2022) introduced the LineVD method, which employs transformer-based models for encoding and leverages graph neural networks (GNNs) to handle control and data dependencies between statements. Liu et al. (2025) extracted feature sets from AST and CFG, significantly enhance vulnerability detection capabilities.

2.3.3 LLM based vulnerability detection

In recent years, with the emergence of highly intelligent LLMs, a variety of methods for vulnerability detection using LLMs have emerged. These methods can be broadly categorized into four main approaches: prompt engineering-based methods, domain-specific training-based methods, multi-agent-based methods, and methods combining with static analysis.

Prompt engineering-based approaches primarily leverage the integration of multiple code representation methods or domain-specific knowledge related to vulnerabilities within prompts to enhance the accuracy and efficiency of LLMs in vulnerability detection tasks. These methods have garnered significant attention and represent the most extensively researched approaches in this domain. Zhang et al. (2024) proposed a method that incorporates source code's data flow information and API invocation sequence information into prompts to improve LLM performance in vulnerability detection tasks. Cao and Jun (2024) integrated CWE names into prompts to assist LLMs in classifying vulnerabilities in the code under analysis. Furthermore, they employed RAG technology to highlight potential vulnerability types and provide example code in the prompt, enabling LLMs to perform the final vulnerability detection task. Lu et al. (2024) introduced the GRACE model, which includes two additional pieces of information in the prompt: one part consists of graph structural information extracted from the code, and the other part leverages RAG technology to retrieve similar code snippets and related domain-specific knowledge from code repositories. By incorporating these two components into the prompt, the LLM is better equipped to perform vulnerability analysis tasks.

Domain-specific training approaches can be further divided into two main categories: training from scratch and fine-tuning on existing models. Wen et al. (2024) proposed a novel approach by representing code as structured natural language comment trees, followed by training a vulnerability detection model based on these structured representations. Guo et al. (2024) conducted a comprehensive study that first evaluated the performance of six commonly used LLMs on vulnerability detection tasks. Subsequently, they fine-tuned the six baseline models, demonstrating the effectiveness of fine-tuning for domain-specific tasks.

Multi-agent-based approaches represent an emerging direction, with their core lying in the task decomposition and coordination between agents. Cao and Jun (2024) proposed a two-agent strategy, dividing the vulnerability detection task into two parts: the vulnerability detection LLM is responsible for analyzing the source code and deriving potential CWE

vulnerability numbers, while the vulnerability analysis LLM further examines whether the source code under inspection indeed contains these vulnerabilities based on the provided information. Wen (2024) introduced a more complex multi-agent architecture, expanding the granularity of vulnerability detection from the function level to the library level. His designed system consists of three agents: the retriever, the coordinator, and the evaluator, with each agent handling two rounds of content at both the function and library levels.

The approach of integrating with static analysis is a common and straightforward method, with the key challenge lying in effectively combining the two techniques. Li et al. (2023) adopted a simple yet effective strategy by re-analyzing code with “uncertain" static vulnerability analysis results using LLMs, thereby reducing the overall system’s false negative rate. Sun et al. (2024) proposed a more complex integration method. Their approach first performs multi-dimensional filtering and reachability analysis on the code, followed by scenario matching and attribute matching using ChatGPT. If a successful match is achieved, the code is then sent to the static analysis module for further confirmation, ultimately yielding the analysis results.

3 Methodology

Preliminary research indicates that integrating highly relevant instances can enhance the prediction accuracy of LLMs (Ma et al., 2023; Luo et al., 2023a). Consequently, we have developed a multi-round approach that integrates both code and its associated vulnerabilities. This method provides highly differentiated instances and their analysis results for each target code, thereby reducing the bias in the detection results caused by factors unrelated to vulnerabilities (such as code length) in large language models. This approach refocuses the detection process on the code itself. While alleviating hallucination, it also solves the problems of low accuracy and poor balance caused by dataset limitations during pre-training.

Our methodology framework is illustrated in Fig. 3, which encompasses five key components: vulnerability database construction, similar code information retrieval, similar code analysis, target code analysis, and analysis result validation. Vulnerability database construction is used to build a database containing source code, vulnerability details, and source code vectors. Similar code retrieval is used to extract similar code and its associated vulnerability information from the database. Similar code analysis is used to generate an analysis report on the robustness or vulnerability points of the similar code. Target code analysis is used to generate a vulnerability analysis report for the target code based on the vulnerability analysis report of the similar code. Analysis validation is used to conduct further validation using the inherent vulnerability detection capabilities of LLMs when the conclusion of the target code is “no vulnerabilities”.

3.1 Vulnerability database construction

In this study, we utilize the large-scale open-source code dataset Big-Vul[1] as the raw data for constructing a vulnerability database. To ensure the comprehensiveness and systematicness of subsequent analysis, we performed a series of preprocessing and integration operations on the raw data. Specifically, we extracted basic information from each code

[1] https://github.com/ZeoVan/MSR_20_Code_Vulnerability_CSV_Dataset

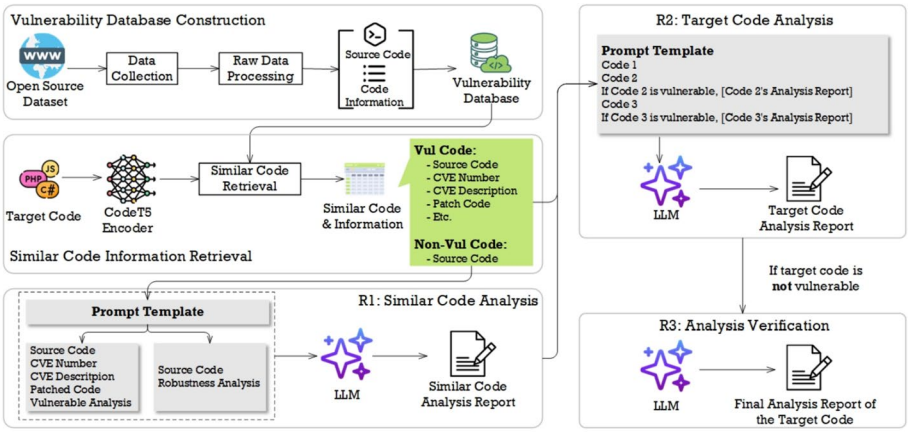

Fig. 3 The framework of DeepVulHunter

segment, including the source code, vulnerability status, and other relevant attributes. For code samples with confirmed vulnerabilities, we further extracted the corresponding Common Vulnerabilities and Exposures (CVE) identifiers, CVE descriptions, and patch codes, and stored this information in a vulnerability database constructed by ourselves to provide data support for the RAG process of subsequent modules.

Meanwhile, to meet the requirements of the RAG in subsequent modules, we need to retrieve the code segments most similar to the target code segment from the database. To achieve efficient similarity retrieval, we used the CodeT5P-110M-Embedding-Model[2] to generate embedding representations for each code segment, converting the source code into feature vectors of dimension 1×256. Subsequently, to eliminate the dimensional differences among the feature vectors of different code segments and improve the accuracy of similarity calculation, we performed L2 normalization on these vectors. Specifically, for a given vector v, we computed its Euclidean norm using (1) and (2).

In (1), we calculate the Euclidean length $\|\mathrm{v}\|_2$ of the vector v by performing the operation of taking the square root of the sum of the squares of the components v_i $(i = 1, 2, \cdots, n)$ of each dimension of the vector v.

$$||\mathrm{v}||_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2} \tag{1}$$

And then in (2) divided each component of the vector v by its Euclidean norm $\|\mathrm{v}\|_2$ to obtain the normalized vector v'.

$$\mathrm{v}' = \frac{\mathrm{v}}{|\mathrm{v}|_2} \tag{2}$$

Finally, we stored these feature vectors in a Faiss index as part of our vulnerability database. After systematic data processing and feature extraction, we constructed a vulnerability database containing function-level code samples. This database not only stores the embedding vectors of each code segment but also records the vulnerability status, CVE identifier,

[2] https://huggingface.co/Salesforce/codet5p-110m-embedding

CVE description, and patch code (with the last three fields being empty if the code is not vulnerable).

3.2 Similar code information retrieval

For the input source code, we adopt the same processing pipeline used during the construction of the database to achieve fast, efficient, and accurate similarity retrieval. Specifically, we first utilize the CodeT5P-110M-Embedding-Model to generate embedding representations of the source code, resulting in 1×256 dimensional feature vectors. Subsequently, we perform L2 normalization on these feature vectors to eliminate the influence of vector length on similarity calculations. After normalization, we compute the dot product between the target vector and all code vectors in the pre-built Faiss index (of type FaissFlatIP). The calculation equation for the dot product between two vectors V_1 and V_2 is shown in (3), where V_{1i} and V_{2i} represent the components of vectors V_1 and V_2 in each dimension respectively:

$$\mathbf{V_1} \cdot \mathbf{V_2} = \sum_{i=1}^{n} V_{1i} \cdot V_{2i} \tag{3}$$

Based on the dot product values, we retrieve the top two code snippets with the highest values as the most similar candidate codes to the target source code. Finally, we extract relevant information from the vulnerability database for each candidate code, including whether it contains a vulnerability, the corresponding CVE identifier, CVE description, and patch code, and pass this information to subsequent analysis stages to support the in-depth advancement of vulnerability detection tasks.

3.3 Similar code analysis

To better support the analysis of the target source code, we draw inspiration from the CoT concept and adopt an iterative fusion method. This approach involves decomposing the analysis of similar code into independent modules, generating structured analysis reports. Such a design aims to enhance the accuracy and robustness of LLMs in the analysis of similar code. Additionally, given that the information structures of vulnerability-free code and vulnerable code stored in the database differ, we have designed dedicated prompt templates for each category of code to accommodate these structural differences.

For the similar code with vulnerabilities, we construct the prompt P_1 as follows:

P_1: You are an expert in source code vulnerability analysis. Below is provided the code, associated CVE number, and CVE description, as well as the patched code. It is known that the following code contains vulnerabilities, and the patched code highlights the changes made before and after the patch. Analyze these changed code snippets and discuss how the vulnerability was fixed in conjunction with the specific code.
Code: [Code]
CVE Number: [CVE No] **CVE Description**: [CVE Des]
Patched Code (In the patch code, the // at the beginning of each line of code below the //fix_flaw_line_below does not indicate a comment, but rather signifies that this is newly added code): [Patched Code]

First, we define the role of a security expert for the LLM to enhance its analytical capabilities and provide it with relevant background information. Based on this, the LLM will identify potential vulnerability points and propose remediation solutions according to the provided information.

We added additional explanations after the “Patched Code" field because the format of the Patched Code is very special and can easily be misinterpreted by LLMs. Here is an example.

```
//flaw_line_below:
unix_notinflight(a);
//fix_flaw_line_below:
//unix_notinflight(a, b);
```

From the above example, it can be observed that the patched code lines are prefixed with “//", which might lead LLMs to mistakenly interpret these lines as comments. This could result in the LLM erroneously deeming the patched code as invalid while considering the unpatched code as valid. To address this, our prompt explicitly specifies: “The ‘//’ preceding the lines below //fix_flaw_line_below does not indicate comments but serves to identify the patched code.” This design aims to prevent LLMs from misinterpreting the “//” symbols and thereby avoid analytical biases in their evaluations.

For similar code without vulnerabilities, the prompt P_2 we constructed is as follows:

P_2: As an expert in source code vulnerability analysis, you are given the following code. It is known that the code below is free of vulnerabilities. Please analyze why the following code is robust.
Code: [Code]

Similar to P_1, we first confirmed the role of the LLM as a security expert to enhance its capabilities. Subsequently, we provided only the source code and instructed the LLM to assess its robustness.

Ultimately, this approach yielded analysis reports for the two most analogous code snippets.

To better illustrate the aforementioned process, we provide a concrete example featuring two similar code segments—one containing a vulnerability and the other being secure. The input, intermediate results, and output of this example are depicted in Fig. 4.

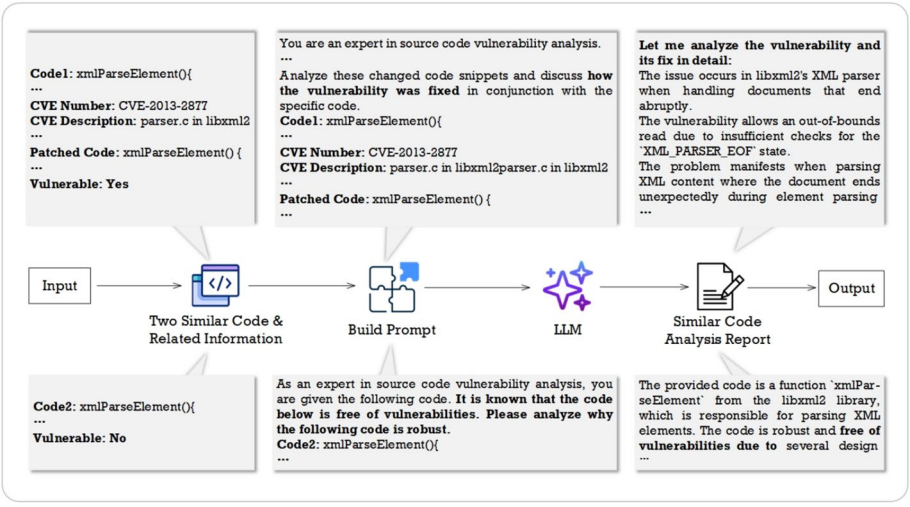

Fig. 4 An example of similar code analysis module. Code1 and Code2 represent the two retrieved code segments that are similar to the target code

In this example, the input consists of two code snippets similar to the target code along with their corresponding vulnerability information. During the prompt construction phase, we integrate this information into the predefined template described earlier. The assembled prompt is then processed by the LLM for analysis, ultimately generating and storing two diagnostic reports.

3.4 Target code analysis

Subsequently, we employ similar code and its analysis report to help LLM perform vulnerability detection on the target code, utilizing the prompt template denoted as P_{3_s} and P_{3_u}.

P_{3_s}: You are a vulnerability detection expert. Your report must end with either 'Code1 vulnerable: yes' or 'Code1 vulnerable: no' based on the analysis results.

P_{3_u}: As a software vulnerability reviewer, you need to produce a vulnerability analysis report for Code1 based on the vulnerability analysis reports of Code2 and Code3 (Code2 and Code3 are structurally similar to Code1). Note that, in addition to analyzing whether Code1 may have the same vulnerabilities as Code2 and Code3, you can also report any potential vulnerabilities you discover. The report must conclude with a determination of whether "Code1 vulnerable: yes" or "Code1 vulnerable: no".
Code1: [Code1]
Code2: [Code2]
If Code2 have vulnerability: [Target1]
Vulnerability analysis reports of Code2: [Code2 Report]
Code3: [Code3]
If Code3 have vulnerability: [Target2]
Vulnerability analysis reports of Code3: [Code3 Report]
The report must end with either "Code1 vulnerable: yes" or "Code1 vulnerable: no" based on the analysis results.

System prompt serving as a core instruction that implicitly guides and controls the model's behavior. We defined system prompt P_{3-s} with two purposes: first, we emphasized the identity of the LLM as a vulnerability detection expert to enable more precise output of domain knowledge, and second, we standardized its output format to facilitate result statistics.

User prompt refers to the direct instructions or queries input by users to LLM, designed to trigger the model to generate specific responses. Unlike the system prompt, it operates at the explicit interaction layer, directly influencing the content of individual outputs. In user prompt P_{3-u}, $\mathrm{Code1}$ represents the target code, while $\mathrm{Code2}$ and $\mathrm{Code3}$ are the most similar codes. Akin to P_1 and P_2, initially, the LLM is designated as a security expert to bolster its capabilities. Subsequently, it evaluates the target code's vulnerability by analyzing the two similar codes, their vulnerability status, and their respective analysis reports. The output is formatted as "Code1 vulnerable: yes" or "Code1 vulnerable: no" for systematic result analysis.

To enhance comprehension of the aforementioned methodology, we present a concrete example illustrating the complete workflow, with corresponding input, intermediate processing stages, and final output depicted in Fig. 5.

The example begins with the input comprising the target code segment alongside two analogous code fragments, each accompanied by their respective vulnerability status (indicating presence or absence of vulnerabilities) and detailed vulnerability analysis reports. During the prompt engineering phase, these components are systematically integrated into the predefined template framework previously established. This composite prompt is subsequently subjected to analysis by the LLM, which generates a comprehensive vulnerability assessment report for the target code. The report culminates in an explicitly stated conclusion regarding the target code's vulnerability status, following which the complete analysis is archived for future reference.

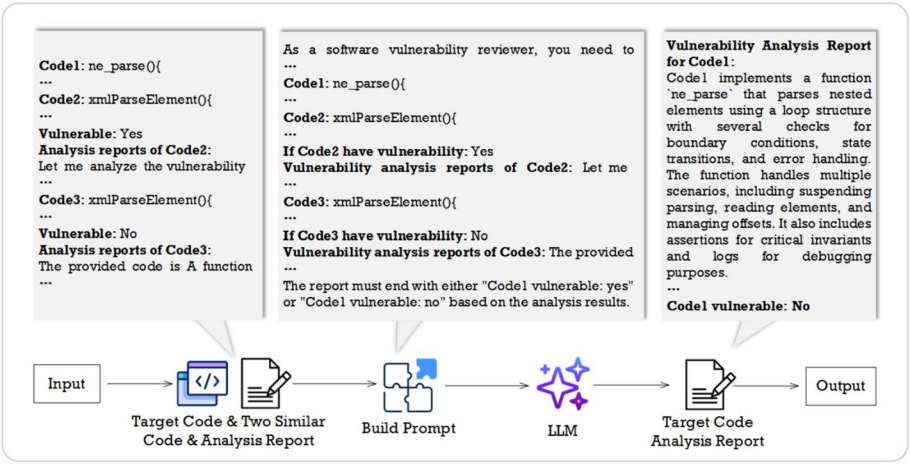

Fig. 5 An example of target code analysis module. Code1 represents the target code. Code2 and Code3 represent the two retrieved code segments that are similar to the target code

3.5 Analysis verification

Statistical analysis reveals that our database contains 177,746 code segments (94.23%) exhibiting no vulnerabilities, while only 10,891 segments (5.77%) display vulnerability. This significant preponderance of non-vulnerable code increases the likelihood that the two most similar codes to any given code will also be non-vulnerable. However, experimental findings indicate that this imbalance can lead to potential misclassification, particularly in the detection of vulnerable code. To mitigate this bias, we have implemented a validation mechanism. This module re-evaluates all code snippets initially classified as "vulnerable: no" by leveraging the LLM's intrinsic capabilities for a comprehensive reassessment of potential vulnerabilities. The outcomes from this secondary evaluation are then integrated to form the final determination. The prompt template for this process is designated as P_{4_s} and P_{4_u}.

> **P_{4_s}**: You are a vulnerability detection expert. Your report must end with either 'Code1 vulnerable: yes' or 'Code1 vulnerable: no' based on the analysis results.

> **P_{4_u}**: You are a vulnerability detection expert. Please carefully analyze the following code and identify if it has any potential security vulnerabilities based on your existing knowledge.
> **Code1**:[Code1]
> Your report must end with either 'Code1 vulnerable: yes' or 'Code1 vulnerable: no' based on the analysis results.

To align with the objectives outlined in Section 3.4, we have retained the identical system prompts. Within the user prompts, we encourage the LLM to leverage its existing knowledge to identify any potential security vulnerabilities in the target code. Additionally, we require the output to adhere to the same structured format as specified in Section 3.4.

To further elucidate the aforementioned methodology, we present a concrete example with its corresponding input, intermediate results, and output as illustrated in Fig. 6. This particular example was selected based on the condition where R2's conclusion returns "Vulnerable: No" - had the output been "Vulnerable: Yes", the system would have directly adopted R2's analysis as the final output without further processing.

The workflow initiates with the input of R2's preliminary analysis report for the Target Code. The system first evaluates the report's conclusion: if designated as "Vulnerable: No", it proceeds to construct a composite prompt by integrating the report's contents into the predefined template framework. This engineered prompt is subsequently processed by the LLM for deeper analysis, ultimately generating the final vulnerability assessment report for the Target Code. Conversely, should the initial conclusion indicate "Vulnerable: Yes", the system immediately treats R2's output as the definitive report. Upon completion, the system systematically archives the finalized vulnerability analysis report for persistent storage and future reference.

4 Experimental setup

4.1 Research questions

To evaluate the effectiveness of our method, we set the following five research questions:

RQ1: how does our method perform compared to other vulnerability detection methods? We aim to investigate whether the ability of DeepVulHunter to detect vulnerabilities in target code can be enhanced by leveraging similar code and vulnerability analysis reports. In this RQ, we will compare DeepVulHunter with four state-of-the-art methods, namely LLMVD (Zhang et al., 2024), CFGNN (Zhang et al., 2023a), GRACE (Lu et al., 2024), and VulACLLM(Liu et al., 2025), to determine whether our approach outperforms these baseline models.

RQ2: how does the performance of our method compare to the vulnerability detection based solely on the LLM itself? We aim to examine whether our approach can genuinely enhance the effectiveness of LLMs in vulnerability detection. In this RQ, we will conduct a

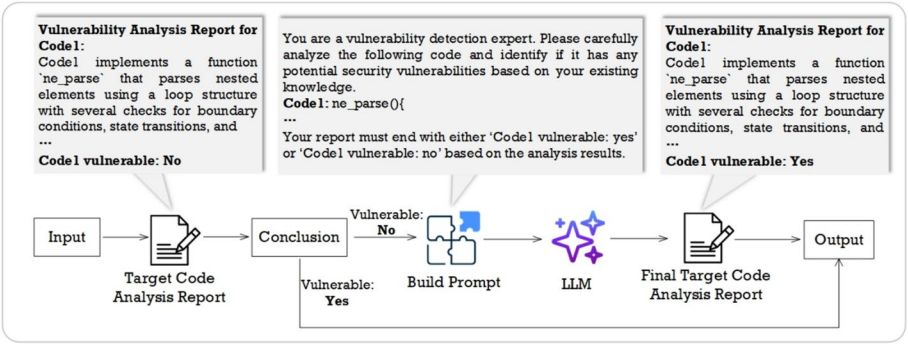

Fig. 6 An example of analysis verification module. Code1 represents the target code

detailed comparison and analysis between the detection results obtained after applying our method and those relying solely on the inherent capabilities of LLMs.

RQ3: after adding the third round, how does the performance compare to using only the first two rounds? We aim to evaluate whether incorporating the analysis validation described in Section 3.5 can enhance the effectiveness of DeepVulHunter. In this RQ, we will examine the results with and without this step in detail.

RQ4: can our approach mitigate bias induced by non-vulnerability factors? Our objective is to investigate whether our method alleviates the outcome bias stemming from code length as identified in Section 1. For this RQ, we will compute the differential in FP and FN ratios between long and short functions after applying our approach, followed by comparative analysis against results generated solely by the LLM baseline.

RQ5: what are the differences in performance among different models? Different LLMs vary in their capabilities due to distinctions in architectural designs, training datasets, and learning objectives. Consequently, their contributions to DeepVulHunter's performance also differ. In this RQ, We will evaluate the performance of each model we used, and analyze the potential primary causes contributing to this discrepancy.

4.2 Dataset

We utilize the test set released in a recent study (Zhang et al., 2024), which is collected from the National Vulnerability Database (NVD)[3]. The NVD encompasses vulnerabilities in software products (i.e., software systems) and may also include different files that describe the differences between vulnerable code and its patched versions. They conduct further extraction, primarily focusing on the '.c' or '.cpp' files that contain vulnerabilities (corresponding to CVE IDs) or their patched versions. Eventually, a test set consisting of 1,937 code snippets is formed, including 1,015 vulnerable code snippets and 922 non-vulnerable code snippets. The distribution of CWE vulnerabilities in the test set is illustrated in Fig. 7, encompassing a total of 38 CWE types. Among these, CWE-119 (Buffer Errors) constitutes the most prevalent category with 371 vulnerable code instances, accounting for 19.2% of the total dataset. Conversely, CWE-193 (Off-by-One Errors) represents the rarest classification, containing only a single documented case. Additionally, a portion of the code lacks associated CWE information. The diversity of vulnerability types ensures that the generalization capability of the method can be thoroughly validated.

It should be specifically noted that during the code retrieval process, when the similarity between retrieved code and target code reaches 100%, the system recursively selects the next candidate in the ranking sequence until all retrieved code segments demonstrate less than 100% similarity to the target code. This mechanism ensures that similar code extracted from our vulnerability database - containing 188,637 annotated code segments with vulnerability labels - maintains sufficient differentiation from the target code, thereby guaranteeing the scientific validity of experimental results.

[3] https://nvd.nist.gov/

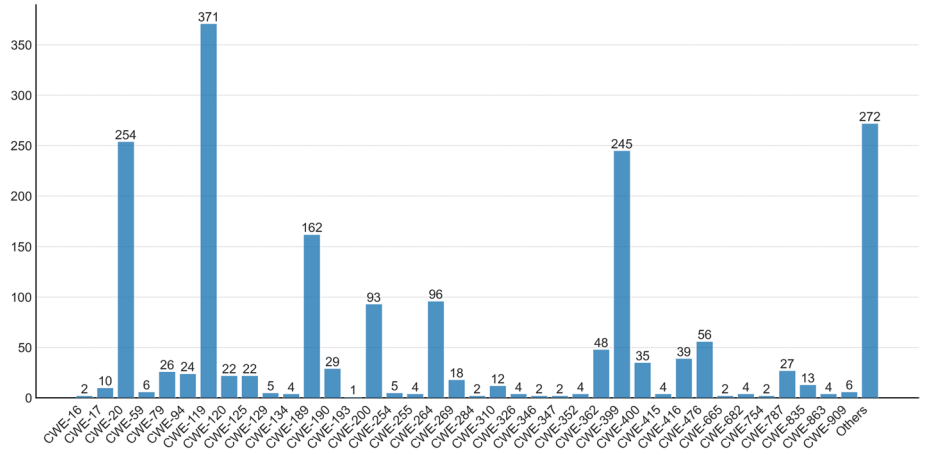

Fig. 7 The distribution of vulnerability types in the test set

4.3 Baselines

We compare our method with four state-of-the-art approaches:

- **LLMVD** (Zhang et al., 2024) is a leading method for vulnerability detection based on LLMs. It enhances ChatGPT's capability to detect vulnerabilities by leveraging API call sequences and DFG.
- **CFGNN** (Zhang et al., 2023a) is a conditional error detection method that automatically identifies condition-related errors using a Control Flow Graph-based Graph Neural Network (CFGNN) and API knowledge.
- **GRACE** (Lu et al., 2024) is a vulnerability detection framework that enhances LLMs by integrating graph structural information (e.g., CFG/AST) and multi-criteria demonstration retrieval for in-context learning.
- **VulACLLM** (Liu et al., 2025) is a vulnerability detection framework that enhances Transformer-based LLMs by integrating combined feature sets extracted from ASTs and CFGs for source-code-level analysis.

4.4 Evaluation metrics

To scientifically and comprehensively evaluate the effectiveness of DeepVulHunter in vulnerability detection, this study employs four commonly recognized and utilized metrics in the field of software engineering for vulnerability detection, i.e., Accuracy , Precision, Recall, and F1-score. These metrics are also applied to the baseline models mentioned in this article, to facilitate comparative analysis.

The performance metrics of the model are defined based on true positives (TP), true negatives (TN), FP, and FN. Specifically, TP occurs when the model accurately identifies a vulnerability in the code. TN is when the model correctly classifies code that does not contain a vulnerability. FP happens when the model incorrectly labels benign code as contain-

ing a vulnerability. Lastly, FN occurs when the model fails to detect an actual vulnerability present in the code. Below are detailed descriptions of these four metrics.

Accuracy refers to the proportion of correctly classified samples out of the total samples, reflecting the overall correctness of the model's classification. This metric comprehensively considers both true positives and true negatives, providing an overall evaluation of the model's performance across all samples. It is calculated using the (4):

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{4}$$

Precision measures the proportion of samples predicted as positive that are actually positive. High precision indicates that the model has a low false positive rate in identifying vulnerabilities and can accurately pinpoint code with actual vulnerabilities. It is calculated using the (5):

$$\text{Precision} = \frac{TP}{TP + FP} \tag{5}$$

Recall , also known as sensitivity, measures the proportion of actual positive samples that are correctly identified by the model, i.e., the ratio of actual vulnerabilities detected to all existing vulnerabilities. High recall indicates that the model has a strong capability in detecting vulnerabilities, enabling it to identify as many actual vulnerabilities in the code as possible. It is calculated using the (6):

$$\text{Recall} = \frac{TP}{TP + FN} \tag{6}$$

F1-score is the harmonic mean of precision and recall, serving as a comprehensive metric to evaluate model performance. It balances precision and recall, yielding a higher F1-score when both precision and recall are high. It provides a more comprehensive assessment of the model's performance in vulnerability detection tasks, avoiding the potential biases that may arise from using precision or recall alone. It is calculated using the (7):

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{7}$$

By applying the above four metrics, we can evaluate the experimental results from multiple perspectives, thereby providing a comprehensive, in-depth, and intuitive description of the effectiveness of our method.

4.5 Implementation

The experimental code was implemented in Python, with database storage and similarity-based retrieval facilitated by the faiss and pandas packages. This study employed five models: Deepseek-V3, Deepseek-R1, Llama-3.1-8B, Llama-3.1-70B, and Llama-3.1-405B.

Both the Deepseek API[4] and Llama API[5] were accessed directly through their official platforms without additional hyperparameter configuration, utilizing each model's default settings. Furthermore, to address the inherent stochasticity and uncertainty in large language model outputs, we conducted five independent experimental trials and computed their averaged results.

5 Experimental results

5.1 RQ1: how does our method perform compared to other vulnerability detection methods?

To investigate this problem, we conducted experiments comparing our method with baselines on the same dataset. The experimental results are presented in Table 1.

Table 1 provides an overview of the detection performance for vulnerable and non-vulnerable code across different methods and models. It is particularly noteworthy that for the LLMVD method -which reports experimental results under multiple distinct prompts - we selected the prompt outcome with the highest accuracy as the control counterpart for each category, regardless of CoT usage.

The experimental results demonstrate that two out of the three top-performing models in terms of accuracy - LLMVD (CoT) and Deepseek-R1 - incorporate CoT methodology, indicating that CoT can significantly enhance model performance.

> **Conclusion 1:** The application of CoT can significantly enhance the vulnerability detection ability of LLMs.

As shown in Table 1, the proposed method demonstrates significant advantages in vulnerability detection tasks. Specifically, our enhanced Llama-405B model achieves the high-

Table 1 Performance comparison between our method and other methods (best results in **bold**), where Vul denotes vulnerable code segments, Non-Vul denotes non-vulnerable code segments, Pre indicates Precision, Rec indicates Recall, F1 indicates F1-score, and Acc indicates Accuracy

Model	Vul			Non-Vul			Total
	Pre	Rec	F1	Pre	Rec	F1	Acc
CFGNN	**1.000**	0.105	0.191	0.393	**1.000**	0.564	0.433
VulACLLM	0.000	0.000	0.000	0.520	**1.000**	**0.690**	0.520
GRACE	0.403	0.587	0.477	0.360	0.211	0.266	0.390
LLMVD(no CoT)	0.525	0.970	0.682	0.516	0.035	0.065	0.525
LLMVD(CoT)	0.675	**0.973**	**0.797**	**0.943**	0.485	0.640	0.741
Llama-8B	0.556	0.873	0.679	0.621	0.229	0.335	0.567
Llama-70B	0.585	0.870	0.700	0.689	0.318	0.435	0.608
Llama-405B	0.709	0.832	0.766	0.770	0.623	0.689	**0.753**
Deepseek-V3	0.707	0.744	0.725	0.701	0.659	0.679	0.704
Deepseek-R1	0.687	0.836	0.754	0.759	0.575	0.654	0.713

[4] https://www.deepseek.com/

[5] https://www.llama.com/

est accuracy of 75.3%, outperforming the LLMVD approach with ChatGPT-4 and CoT by 1.2%. When excluding all CoT-based methods (LLMVD and Deepseek-R1), our method exhibits a maximum improvement of 22.8% and an average advantage of 13.3% over the best-performing baseline model (GPT-4-based LLMVD). Further, the absolute differences in F1-scores between vulnerable and non-vulnerable samples for Deepseek-V3, Llama-405B, and Deepseek-R1 enhanced by our method are 4.6%, 7.7%, and 10% respectively, all significantly lower than the lowest baseline model LLMVD with CoT (15.7%). These results conclusively prove that our method effectively addresses the detection imbalance between positive and negative samples while maintaining high detection accuracy in vulnerability detection tasks.

Conclusion 2: Compared with other advanced vulnerability detection methods, our method boasts better accuracy. It enables the Llama-405B model to outperform ChatGPT in terms of vulnerability detection capabilities. In terms of balance, our method also demonstrates outstanding performance.

5.2 RQ2: how does the performance of our method compare to the vulnerability detection based solely on the LLM itself?

To investigate this issue, we had LLMs generate analysis results independently on the same dataset, processed and summarized these results, and then conducted a horizontal comparison with the experimental results obtained using our method. The comparison results are presented in Table 2. It is worth noting that, to further control variables, we applied exactly the same prompt as in R3. However, the difference from R3 is that in this round, we asked the LLMs to analyze all the code snippets in the dataset, while in R3, only the code snippets with the result of "Vulnerable: no" from R2 were analyzed.

Table 2 Performance comparison between our method and the baseline LLM (best results in **bold**), where Base denotes results using only the LLM's native capabilities, R3 shows results with DeepVulHunter, Vul indicates vulnerable code segments, Non-Vul indicates non-vulnerable code segments, Pre indicates precision, Rec indicates recall, F1 indicates F1-score, and Acc indicates accuracy

Model	Base						
	Vul			Non-Vul			Total
	Pre	Rec	F1	Pre	Rec	F1	Acc
Llama-8B	0.573	0.611	0.591	0.547	0.507	0.526	0.561
Llama-70B	0.587	0.797	0.676	0.647	0.398	0.493	0.604
Llama-405B	0.689	0.742	0.715	0.700	0.643	0.670	0.694
Deepseek-V3	**0.764**	0.618	0.683	0.652	**0.789**	**0.714**	0.700
Deepseek-R1	0.722	0.688	0.705	0.667	0.703	0.685	0.695
	R3						
Llama-8B	0.556	**0.873**	0.679	0.621	0.229	0.335	0.567
Llama-70B	0.585	0.870	0.700	0.689	0.318	0.435	0.608
Llama-405B	0.709	0.832	**0.766**	**0.770**	0.623	0.689	**0.753**
Deepseek-V3	0.707	0.744	0.725	0.701	0.659	0.679	0.704
Deepseek-R1	0.687	0.836	0.754	0.759	0.575	0.654	0.713

The experimental results demonstrate that, in terms of accuracy, our method exhibits consistent improvements across all models compared to relying solely on the LLMs themselves, with a maximum improvement of 5.3% and an average enhancement of 1.82% observed across the five models. Notably, the accuracy differences between the Base and R3 versions were relatively minor for the Llama-70B and Deepseek-V3 models. We hypothesize that this limited improvement stems from the well-balanced detection performance achieved by our carefully constructed Base prompts. During the R1 and R2 phases, these models misclassified certain vulnerable code segments as non-vulnerable. While the R3 phase corrected these errors, the magnitude of corrections essentially offset the potential improvements, resulting in only marginal performance gains.

Conclusion 3: With the application of our method, the accuracy of LLMs in vulnerability detection has been enhanced compared to relying solely on their inherent capabilities, which demonstrates the effectiveness of our method.

5.3 RQ3: after adding the third round, how does the performance compare to using only the first two rounds?

To investigate this issue, we stored and processed the intermediate experimental results obtained from different models after applying only R1 and R2 rounds, and conducted a horizontal comparison with the results after applying R3 rounds. The comparison results are presented in Table 3.

It can be analyzed that after applying R3, the accuracy of all models has been improved compared to when R3 was not applied. The improvement ranges from 3.8% to 18.9%. The mean improvement is 9.8%. This conclusively demonstrates the necessity of the R3 module. Further, after applying R3 to Llama-8B, Llama-70B and Deepseek-R1, the absolute value of

Table 3 Performance comparison of our method before and after incorporating R3 (best results in **bold**), where R2 denotes results without the analysis validation module described in Section 3.5, R3 represents DeepVulHunter's performance after complete three-round analysis, Vul indicates vulnerable code segments, Non-Vul indicates non-vulnerable code segments, Pre indicates precision, Rec indicates recall, F1 indicates F1-score, and Acc indicates accuracy

Model	R2						
	Vul			Non-Vul			Total
	Pre	Rec	F1	Pre	Rec	F1	Acc
Llama-8B	0.543	0.656	0.594	0.505	0.389	0.439	0.529
Llama-70B	0.613	0.408	0.490	0.523	0.716	0.604	0.554
Llama-405B	**0.715**	0.282	0.404	0.525	**0.876**	0.657	0.564
Deepseek-V3	0.703	0.405	0.514	0.553	0.812	0.658	0.598
Deepseek-R1	0.684	0.474	0.560	0.565	0.757	0.647	0.608
	R3						
Llama-8B	0.556	**0.873**	0.679	0.621	0.229	0.335	0.567
Llama-70B	0.585	0.870	0.700	0.689	0.318	0.435	0.608
Llama-405B	0.709	0.832	**0.766**	**0.770**	0.623	**0.689**	**0.753**
Deepseek-V3	0.707	0.744	0.725	0.701	0.659	0.679	0.704
Deepseek-R1	0.687	0.836	0.754	0.759	0.575	0.654	0.713

the difference in F1-scores between Vul and Non-Vul samples has increased, which means that R3 has made the results more unbalanced. However, for other models, the absolute value of the difference in F1-scores between Vul and Non-Vul samples has decreased, indicating that R3 has made their results more balanced. The inference about the decrease in the balance of Llama-8B and Llama-70B has been elaborated in Section 5.2. For Deepseek-R1, it can be seen that its significant difference from Llama-8B and Llama-70B is that the F1-scores of both Vul and Non-Vul samples of the latter have increased compared to R2, only with different improvement amplitudes. Therefore, the decrease in balance does not conflict with the improvement in performance, mainly because the model itself is more inclined to detect code as vulnerable.

Conclusion 4: The application of R3 can improve the overall accuracy. Therefore, the verification process of the analysis results is meaningful. However, its impact on the balance varies across different models.

5.4 RQ4: can our approach mitigate bias induced by non-vulnerability factors?

To investigate this issue, we visualized the ratio of FP to FN in long and short functions after using our proposed method (Base phase results are shown in Fig. 2 of Section 1). The comparative results are presented in Fig. 8. We specifically examined whether the disparity in FP and FN ratios between long and short functions (represented by the distance between the blue-green boundaries in the bar charts) was reduced. As detailed in Table 4, our analysis reveals that for all models except Deepseek-R1, the FP and FN ratio disparity decreased by 4.6%-6.5% after applying our method compared to the baseline. This statistically significant reduction demonstrates our method's effectiveness in mitigating bias and inaccuracies caused by code length in non-reasoning models.

Conclusion 5: Our method effectively mitigates the imbalance and inaccuracy in detection results caused by code-length.

5.5 RQ5: what are the differences in performance among different models?

From the Base experiment results in Table 2, it can be observed that the average accuracy of the Deepseek series is 69.8%, whereas the Llama series has an average accuracy of 62.0%, indicating that the Deepseek series outperforms the Llama series. Furthermore, the average F1-score differences between the two sample types are 0.092 for the Deepseek series and 0.026 for the Llama series. This disparity may be attributed to the phrase 'any potential security vulnerabilities' in the prompts, suggesting that the Deepseek series is less sensitive to such potential cues and focuses more on the problem at hand.

Conclusion 6: The Deepseek series demonstrates superior vulnerability detection capabilities compared to the Llama series and exhibits a more 'rational' approach.

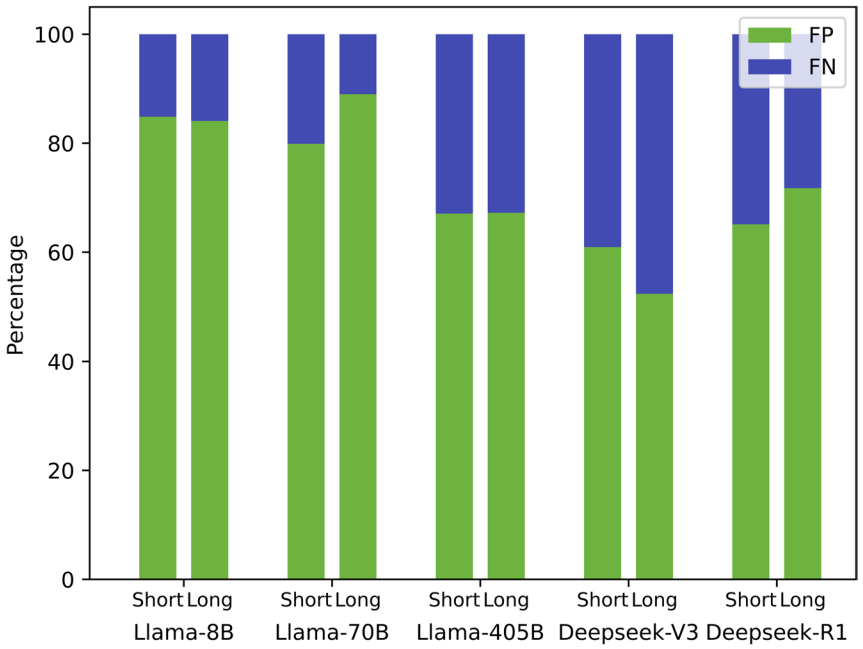

Fig. 8 Telative proportions of FP and FN in short and long functions misclassified by different models in R3. A greater distance between the blue-green boundaries indicates more severe interference from code length

Table 4 The absolute difference in FP and FN ratios between long and short functions in Base and R3 phases, and the difference between these absolute differences

Model	Phase		
	Base	R3	Δ
Llama-8B	0.073	0.008	0.065
Llama-70B	0.155	0.091	0.064
Llama-405B	0.056	0.001	0.055
Deepseek-V3	0.132	0.086	0.046
Deepseek-R1	0.002	0.066	-0.064

Base and R3 represent the difference in FP and FN ratios between long and short functions (measured by the distance between blue-green boundaries) in their respective phases, while Δ denotes the difference between these two phases. A larger Δ value indicates our method's stronger capability to mitigate bias caused by code length

Through an extensive literature review, we attribute this phenomenon to synergistic factors encompassing architectural design, training data, and optimization strategies. Specifically, Deepseek models employ Multi-head Latent Attention (MLA) with low-rank compression to achieve adaptive information filtering, effectively suppressing redundant signals from prompts (e.g., modifiers like "potential"). This mechanism, combined with sparse activation in their Mixture-of-Experts (MoE) architecture (671B total parameters with 37B activated per token), dynamically mitigates irrelevant interference. In contrast, Llama's dense architecture lacks such active noise suppression capabilities, rendering it more vulnerable to prompt perturbations (Touvron et al., 2023; Guo et al., 2025; Liu et al., 2024).

Furthermore, Deepseek's training regimen – characterized by high-proportion STEM (Science, Technology, Engineering, and Mathematics) and code datasets (e.g., MATH benchmarks, HumanEval) and a four-stage training pipeline – cultivates strong inductive biases toward logical tasks, reducing dependency on ambiguous prompts. Enhanced anti-hallucination optimization further reinforces objective semantic parsing. These design principles collectively enable Deepseek to prioritize objective task components while exhibiting reduced susceptibility to biased lexical cues in prompts.

6 Threats to validity

Our methodology currently exhibits several limitations and potential directions for future research, as outlined below:

The current methodology is exclusively tailored for C/C++ codebases, with mainstream languages such as Java and Python not yet incorporated into the evaluation framework. Future extensions should expand the database and test suites to validate efficacy across additional programming languages.

Generating comprehensive reports for all model outputs incurs substantial temporal and financial costs while complicating result processing. Future implementations could optimize efficiency by restricting R2 and R3 stages to binary classification outcomes (0/1).

The test set employed does not encompass all CWE types, potentially introducing result volatility across different vulnerability distributions. Subsequent work should adopt more comprehensive datasets to ensure robust evaluation.

This study does not incorporate all mainstream model architectures, notably excluding the GPT-series models. Expanding the experimental scope to include additional state-of-the-art LLMs would strengthen benchmarking comprehensiveness.

7 Conclusion

The emergence of LLMs has brought transformative changes across various domains. Our research reveals that LLMs' vulnerability detection capabilities are susceptible to interference from non-vulnerability-related factors such as code length, while also suffering from hallucinations, low accuracy, and poor detection balance. To address these challenges, this paper proposes DeepVulHunter – a multi-round detection method leveraging similar code segments and their vulnerability information, designed to fully unleash LLMs' potential in vulnerability detection.

Our experiments have demonstrated that our method can indeed enhance the LLMs' capabilities in the field of vulnerability detection, and its performance outperforms the current state-of-the-art methods. In addition, we have summarized the vulnerability detection capabilities of two series of models based on the experimental process and results.

Acknowledgements This work was supported by National Natural Science Foundation of China (No.62472296), Sichuan Science and Technology Program (No.2024ZHCG0197).

Author Contributions Y.J. and J.H. conceptualized the study, developed the methodology, and conducted the experiments. C.H. supervised the research, provided critical insights, and reviewed the overall study design. All authors wrote and reviewed the main manuscript text.

Data Availability Data will be available on request.

Declarations

Competing interests The authors declare no competing interests.

References

Achiam, J., Adler, S., Agarwal, S., et al. (2023). Gpt-4 technical report. arXiv:2303.08774https://doi.org/10.48550/arXiv.2303.08774

Al Debeyan, F., Hall, T., Bowes, D. (2022). Improving the performance of code vulnerability prediction using abstract syntax tree information. In: Proceedings of the 18th International Conference on Predictive Models and Data Analytics in Software Engineering, pp. 2–11,https://doi.org/10.1145/3558489.3559066

Anil, R., Dai, AM., Firat, O., et al. (2023). Palm 2 technical report. arXiv:2305.10403https://doi.org/10.48550/arXiv.2305.10403

Bommasani, R., Hudson, DA., Adeli, E., et al. (2021). On the opportunities and risks of foundation models. arXiv:2108.07258https://doi.org/10.48550/arXiv.2108.07258.

Brown, T., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems 33*, 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf

Cao, D., Jun, W. (2024). Llm-cloudsec: Large language model empowered automatic and deep vulnerability analysis for intelligent clouds. In: IEEE INFOCOM 2024-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), IEEE, pp. 1–6, https://doi.org/10.1109/INFOCOMWKSHPS61880.2024.10620804

Cao, J., Li, M., Wen, M., et al. (2025). A study on prompt design, advantages and limitations of chatgpt for deep learning program repair. *Automated Software Engineering, 32*(1), 1–29. https://doi.org/10.1007/s10515-025-00492-x

Chen, X., Djolonga, J., Padlewski, P., et al. (2023). Pali-x: On scaling up a multilingual vision and language model. arXiv:2305.18565https://doi.org/10.48550/arXiv.2305.18565

Cheng, X., Zhang, G., Wang, H., et al. (2022). Path-sensitive code embedding via contrastive learning for software vulnerability detection. In: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 519–531, https://doi.org/10.1145/3533767.3534371

Chowdhery, A., Narang, S., Devlin, J., et al. (2023). Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research 24*(240), 1–113. http://jmlr.org/papers/v24/22-1144.html

Cui, L., Hao, Z., Jiao, Y., et al. (2020). Vuldetector: Detecting vulnerabilities using weighted feature graph comparison. *IEEE Transactions on Information Forensics and Security, 16*, 2004–2017. https://doi.org/10.1109/TIFS.2020.3047756

De Kraker, W., Vranken, H., Hommmersom, A. (2023). Glice: combining graph neural networks and program slicing to improve software vulnerability detection. In: 2023 IEEE European Symposium on Security and Privacy Workshops (EuroS &PW), IEEE, pp. 34–41, https://doi.org/10.1109/EuroSPW59978.2023.00009

Gao, Q., Ma, S., Shao, S., et al. (2018). Cobot: static c/c++ bug detection in the presence of incomplete code. In: Proceedings of the 26th Conference on Program Comprehension, pp. 385–388, https://doi.org/10.1145/3196321.3196367

Goud, MD. (2025). Android malware detection using equilibrium optimized deep learning-based pattern recognition. *European Advanced Journal for Emerging Technologies (EAJET)-p-ISSN 3050-9734 en e-ISSN 3050-9742 3*(3). https://doi.org/10.5281/zenodo.15863110

Grieco, G., Grinblat, GL., Uzal, L., et al. (2016). Toward large-scale vulnerability discovery using machine learning. In: Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy, pp. 85–96, https://doi.org/10.1145/2857705.2857720

Guo, D., Yang, D., Zhang, H., et al. (2025). Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv:2501.12948https://doi.org/10.48550/arXiv.2501.12948

Guo Y, Patsakis C, Hu Q, et al (2024) Outside the comfort zone: Analysing llm capabilities in software vulnerability detection. In: European Symposium on Research in Computer Security, Springer, pp 271–289, https://doi.org/10.1145/3607199.3607242

Harer, JA., Kim, LY., Russell, RL., et al. (2018). Automated software vulnerability detection with machine learning. arXiv:1803.04497https://doi.org/10.48550/arXiv.1803.04497

Hin, D., Kan, A., Chen, H., et al. (2022). Linevd: Statement-level vulnerability detection using graph neural networks. In: Proceedings of the 19th International Conference on Mining Software Repositories, pp. 596–607, https://doi.org/10.1145/3524842.3527949

Hoffmann, J., Borgeaud, S., Mensch, A., et al. (2022). Training compute-optimal large language models. arXiv:2203.15556https://doi.org/10.48550/arXiv.2203.15556

Hou, J., Han, J., Huang, C., et al. (2025). Linejlocrepair: A line-level method for automated vulnerability repair based on joint training. *Future Generation Computer Systems, 166*, Article 107671. https://doi.org/10.1016/j.future.2024.107671

International Business Machines. (2021). Cost of a data breach report 2021. *Technical Representative*, IBM, https://www.ibm.com/reports/data-breach

Kaur, A., & Nayyar, R. (2020). A comparative study of static code analysis tools for vulnerability detection in c/c++ and java source code. *Procedia Computer Science, 171*, 2023–2029. https://doi.org/10.1016/j.procs.2020.04.217

Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems 33*, 9459–9474. https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf

Li, H., Hao, Y., Zhai, Y., et al. (2023). Assisting static analysis with large language models: A chatgpt experiment. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 2107–2111, https://doi.org/10.1145/3611643.3613078

Li, Z., Zou, D., Xu, S., et al. (2021). Sysevr: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing, 19*(4), 2244–2258. https://doi.org/10.1109/TDSC.2021.3051525

Lipp, S., Banescu, S., Pretschner, A. (2022). An empirical study on the effectiveness of static c code analyzers for vulnerability detection. In: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 544–555, https://doi.org/10.5281/zenodo.6515687

Liu, A., Feng, B., Xue, B., et al. (2024). Deepseek-v3 technical report. arXiv:2412.19437https://doi.org/10.48550/arXiv.2412.19437

Liu, J., Lin, G., Mei, H., et al. (2025). Enhancing vulnerability detection efficiency: An exploration of lightweight llms with hybrid code features. *Journal of Information Security and Applications, 88*, Article 103925. https://doi.org/10.1016/j.jisa.2024.103925

Liu, P., Yuan, W., Fu, J., et al. (2023). Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys, 55*(9), 1–35. https://doi.org/10.1145/3560815

Liu, V., Chilton, LB. (2022). Design guidelines for prompt engineering text-to-image generative models. In: Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems, pp. 1–23, https://doi.org/10.1145/3491102.3501825

Lu, G., Ju, X., Chen, X., et al. (2024). Grace: Empowering llm-based software vulnerability detection with graph structure and in-context learning. *Journal of Systems and Software, 212*, Article 112031. https://doi.org/10.1016/j.jss.2024.112031

Luo, M., Xu, X., Dai, Z., et al. (2023a). Dr. icl: Demonstration-retrieved in-context learning. arXiv:2305.14128https://doi.org/10.48550/arXiv.2305.14128

Luo, Z., Xu, C., Zhao, P., et al. (2023b). Wizardcoder: Empowering code large language models with evol-instruct. arXiv:2306.08568https://doi.org/10.48550/arXiv.2306.08568

Ma, P., Ding, R., Wang, S., et al. (2023). Demonstration of insightpilot: An llm-empowered automated data exploration system. arXiv:2304.00477https://doi.org/10.48550/arXiv.2304.00477

Maddigan, P., & Susnjak, T. (2023). Chat2vis: Generating data visualizations via natural language using chatgpt, codex and gpt-3 large language models. *IEEE Access, 11*, 45181–45193. https://doi.org/10.1109/ACCESS.2023.3274199

Pang, Y., Xue, X., Namin, AS. (2015). Predicting vulnerable software components through n-gram analysis and statistical feature selection. In: 2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA), IEEE, pp. 543–548, https://doi.org/10.1109/ICMLA.2015.99

Pitis, S., Zhang, MR., Wang, A., et al. (2023). Boosted prompt ensembles for large language models. arXiv:2304.05970https://doi.org/10.48550/arXiv.2304.05970

Powell, C., & Riccardi, A. (2025). Generating textual explanations for scheduling systems leveraging the reasoning capabilities of large language models. *Journal of Intelligent Information Systems, 63*(4), 1287–1337. https://doi.org/10.1007/s10844-025-00940-w

Qin, G., Eisner, J. (2021). Learning how to ask: Querying lms with mixtures of soft prompts. arXiv:2104.06599https://doi.org/10.48550/arXiv.2104.06599

Raihan, N., Goswami, D., Puspo, S., et al. (2025). On the performance of large language models on introductory programming assignments. *Journal of Intelligent Information Systems*. https://doi.org/10.1007/s10844-025-00968-y

Ren, Z., Ju, X., Chen, X., et al. (2025). Improving distributed learning-based vulnerability detection via multi-modal prompt tuning. *Journal of Systems and Software, 226*, Article 112442. https://doi.org/10.1016/j.jss.2025.112442

Scandariato, R., Walden, J., Hovsepyan, A., et al. (2014). Predicting vulnerable software components via text mining. *IEEE Transactions on Software Engineering, 40*(10), 993–1006. https://doi.org/10.1109/TSE.2014.2340398

Shang, J., Li, J., Sui, Y., et al. (2025). Cegt: Smart contract vulnerability detection via connectivity-enhanced gcn-transformer. *Journal of Systems and Software* p. 112454. https://doi.org/10.1016/j.jss.2025.112454

Shar, L. K., Briand, L. C., & Tan, H. B. K. (2014). Web application vulnerability prediction using hybrid program analysis and machine learning. *IEEE Transactions on Dependable and Secure Computing, 12*(6), 688–707. https://doi.org/10.1109/TDSC.2014.2373377

Srivastava, A., Rastogi, A., Rao, A., et al. (2022). Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. arXiv:2206.04615https://doi.org/10.48550/arXiv.2206.04615

Sui, Y., Xue, J. (2016). Svf: interprocedural static value-flow analysis in llvm. In: Proceedings of the 25th International Conference on Compiler Construction, pp. 265–266, https://doi.org/10.1145/2892208.2892235

Sun, Y., Wu, D., Xue, Y., et al. (2024). Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, pp. 1–13, https://doi.org/10.1145/3597503.3639117

Tay, Y., Dehghani, M., Tran, VQ., et al. (2022). Ul2: Unifying language learning paradigms. arXiv:2205.05131https://doi.org/10.48550/arXiv.2205.05131

Touvron, H., Lavril, T., Izacard, G., et al. (2023). Llama: Open and efficient foundation language models. arXiv:2302.13971https://doi.org/10.48550/arXiv.2302.13971

Wen, XC. (2024). Collaboration to repository-level vulnerability detection. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 1926–1928, https://doi.org/10.1145/3650212.3685562

Wen, XC., Gao, C., Gao, S., et al. (2024). Scale: Constructing structured natural language comment trees for software vulnerability detection. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 235–247, https://doi.org/10.1145/3650212.3652124

White, J., Hays, S., Fu, Q., et al. (2024). Chatgpt prompt patterns for improving code quality, refactoring, requirements elicitation, and software design. arxiv 2023. arXiv:2303.07839https://doi.org/10.48550/arXiv.2303.07839

Yang, J., Ruan, O., & Zhang, J. (2024). Tensor-based gated graph neural network for automatic vulnerability detection in source code. *Software Testing, Verification and Reliability, 34*(2), Article e1867. https://doi.org/10.1002/stvr.1867

Ye, H., Liu, T., Zhang, A., et al. (2023). Cognitive mirage: A review of hallucinations in large language models. arXiv:2309.06794https://doi.org/10.48550/arXiv.2309.06794

Zagane, M., Abdi, M. K., & Alenezi, M. (2020). A new approach to locate software vulnerabilities using code metrics. *International Journal of Software Innovation (IJSI), 8*(3), 82–95. https://doi.org/10.4018/IJSI.2020070106

Zeng, A., Liu, X., Du, Z., et al. (2022). Glm-130b: An open bilingual pre-trained model. arXiv:2210.02414https://doi.org/10.48550/arXiv.2210.02414

Zhang, C., Liu, H., Zeng, J., et al. (2024) Prompt-enhanced software vulnerability detection using chatgpt. In: Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, pp. 276–277, https://doi.org/10.1145/3639478.3643065

Zhang, J., Wang, X., Zhang, H., et al. (2023a). Detecting condition-related bugs with control flow graph neural network. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 1370–1382, https://doi.org/10.1145/3597926.3598142

Zhang, S., Roller, S., Goyal, N., et al. (2022). Opt: Open pre-trained transformer language models. arXiv:2205.01068https://doi.org/10.48550/arXiv.2205.01068

Zhang, Y., Li, Y., Cui, L., et al. (2023b). Siren's song in the ai ocean: a survey on hallucination in large language models. arXiv:2309.01219https://doi.org/10.48550/arXiv.2309.01219

Zhao, WX., Zhou, K., Li, J., et al. (2023). A survey of large language models. *1*(2). arXiv:2303.18223https://doi.org/10.48550/arXiv.2303.18223